Disentangling Candidate Breadth and Trajectory Supervision in Execution-Guided Program Repair

ANONYMOUS AUTHOR(S)

Candidate breadth can masquerade as a training-method improvement in execution-guided program repair. We introduce an identification framework and use ZPDPatch as its experimental instrument. ZPDPatch mines three executable relations from user submission trajectories, trains independent QLoRA checkpoints, and selects a validation-frozen three-member portfolio. Its controller provably preserves observed-test pass rate, enforces an optional AST edit budget, and finds a repair whenever the emitted candidate set contains an eligible one. Across 997 Seen and 250 problem-held-out Python trajectories, same-draw decomposition, temperature sweeps, decoding-matched checkpoints, and an untuned-model control separate stochastic breadth from supervision. The results reject a simple heterogeneous-policy account: breadth explains most unrestricted coverage, and five-fold problem cross-fitting finds no conclusive mixed-target advantage over a selection-matched Answer pool. Relation-mined targets instead improve mean coverage across six edit budgets. A current-only three-Answer deployment repairs 72.0/81.2% of Seen/Unseen inputs, while retrieval-based LSGen remains stronger without an edit limit. Hidden-test, difficulty-matching, smaller-model, and Java audits delimit rather than erase these conclusions.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **Applied computing** → *Education*; • **Computing methodologies** → *Natural language generation*.

Additional Key Words and Phrases: automated program repair, programming education, large language models, submission trajectories, execution-guided repair

ACM Reference Format:
Anonymous Author(s). 2027. Disentangling Candidate Breadth and Trajectory Supervision in Execution-Guided Program Repair. In *Proceedings of ACM International Conference on the Foundations of Software Engineering (FSE '27)*. ACM, New York, NY, USA, 21 pages.

1 Introduction

Programmers iteratively write, submit, execute, and revise code. Online judges make this loop observable at scale, but their coarse verdicts do not explain whether a revision progressed or moved away from a viable solution. These submission sequences are a largely unused source of supervision for repair.

Automated program repair (APR) offers executable, code-specific feedback. The field spans search-based generate-and-validate repair, learned transformations, and test-guided validation [Gazzola et al. 2019; Le Goues et al. 2012; Monperrus 2018; Weimer et al. 2009]. Systems for programming assignments cluster peer solutions, align a user's code to a reference, or synthesize a fixed program [Gulwani et al. 2018; Hu et al. 2020; Wang et al. 2018]. More recent systems use language models, execution diagnostics, retrieved examples, and iterative conversations [Joshi et al. 2023; Liu et al. 2024; Orvalho et al. 2025; Yang et al. 2024; Zhang et al. 2024]. Most condition on the latest buggy program, possibly augmented with tests or peer programs; earlier submissions by the same user are rarely first-class repair context.

The motivation for using history is straightforward but unverified. Two users may arrive at similar programs through different revisions; their histories expose attempted edits and states they demonstrably reached. In educational terms, such reachable improvements resemble the Zone of Proximal Development (ZPD) [Chounta et al. 2017; Cole et al. 1978; Murray and Arroyo 2002]. We ask whether repair can exploit an observed trajectory rather than immediately replace the current program with an arbitrary correct solution. ZPD is design motivation, not a measured psychological construct or learning-outcome claim.

We use ZPDPatch, illustrated in Figure 1, as an experimental instrument for mechanism separation rather than presuming trajectory superiority. It automatically converts authentic submission trajectories into three supervised repair datasets. Progress retains verdict improvements and same-verdict transitions whose per-test outcomes improve monotonically. Strict retains only transitions that improve the online-judge verdict. Answer pairs every failed submission with the trajectory's final accepted program. Three seeds per target relation produce nine independently trained QLoRA checkpoints. Validation execution selects three checkpoints; execution and AST tree edit distance (TED) control acceptance and abstention.

Our primary contribution is an empirical identification design for mechanisms that are easy to conflate: output sampling, independent training seeds, validation-selected pool breadth, relation-specific dataset construction, and serialization of prior submissions. The $A_1 \to T_1 \to S_3 \to A_3 \to A_9 \to M_9$ ladder matches each successive opportunity. Target-matched retraining separately finds no stable causal benefit from serializing earlier submissions.

This paper makes the following contributions:

- We design and prove a generator-agnostic execution controller whose fallback, early-stop, and structural gate guarantee observed-test non-regression, candidate-relative repair completeness, and AST edit-budget compliance by construction.
- We formulate trajectory-mined repair as finite execution-evidence orders plus terminal closure, and audit every materialized supervision transition against those definitions.
- We pose checkpoint choice as exact validation-coverage search and derive a same-draw decomposition that measures candidate breadth without attributing ordinary inference scaling to training.
- We empirically separate output, checkpoint, target-construction, and selection diversity using the full A_1–T_1–S_3–A_3–A_9–M_9 ladder, target-matched history ablations, hidden tests, complete five-fold problem-cross-fitted selection, patch-budget and source-retention analyses, a second model scale, two orthogonal Java holdouts, prompt-distribution controls, and model-level verdict-order retraining.

2 Background and Related Work

2.1 Structure- and Reference-Based Educational Repair

Educational APR can exploit many programs written for one specification. CLARA clusters and aligns programs before extracting a repair; SARFGEN searches and aligns reference programs; Refactory normalizes refactorings before comparison [Gulwani et al. 2018; Hu et al. 2020; Wang et al. 2018]. Embeddings, solution-space models, and visualization propagate annotations or expose recurring strategies, while multi-function and diversified methods broaden the target beyond one correction [Choi et al. 2021; Choi and Lee 2025; Glassman et al. 2015; Heo et al. 2023; Piech et al. 2015b; Rivers and Koedinger 2017; Yi et al. 2017].

LSGen is our reference-rich baseline. It filters historical repair pairs, retrieves by edit information, generates, and may retrieve again from the new state [Dai et al. 2026]. We retain its access to other users' Seen-Train solutions for the target problem. That information is unavailable on genuinely new assignments, so LSGen is not evaluated on problem-held-out Unseen tasks. This separates repository-mature repair from repair using only a learned model, statement, and tests.

2.2 Neural and Large-Language-Model Repair

Classical generate-and-validate APR established both the power of broad search and the risk of test-suite overfitting [Le Goues et al. 2012; Qi et al. 2014; Smith et al. 2015; Weimer et al. 2009]. Neural repair learned transformations from repository histories and context-aware translation before pretrained code models enabled zero-shot and fine-tuned repair [Jiang et al. 2023; Lutellier et al. 2020; Tufano et al. 2019; Xia and Zhang 2022; Yasunaga and Liang 2021]. RING, PyDex, and FastFixer cast educational repair as conditional generation [Joshi et al. 2023; Liu et al. 2024; Zhang et al. 2022, 2024]; retrieval and localization further constrain generation with peer programs or high-precision syntax feedback [Phung et al. 2023; Zhao et al. 2024]. These systems cover programs that do not align cleanly with a reference, but make correctness and patch scope external validation concerns.

Execution-aware methods address those concerns with signals independent of model confidence. SelfAPR learns from test diagnostics; FixEval makes execution the repair-evaluation criterion; CREF carries diagnostics through a repair conversation; counterexample-guided repair combines generation, fault localization, and MaxSAT [Haque et al. 2023; Orvalho et al. 2025; Yang et al. 2024; Ye et al. 2022]. ZPDPatch likewise executes candidates, but introduces a different unit of supervision and control. Authentic trajectories define nested improvement relations rather than synthetic corruptions or manually assigned repair roles. Members share a base model and may have correlated errors, but generation is non-adaptive: no member consumes another member's output. The unchanged program remains selectable, so a failed generation cannot force a regression.

2.3 Adaptive Feedback, ZPD, and Submission Histories

Programming feedback ranges from correctness reports to next-step hints and complete repairs; feedback research emphasizes timeliness, task focus, and actionable information rather than correctness alone [Hattie and Timperley 2007; Keuning et al. 2018; Narciss 2008; Shute 2008]. Tutoring research studies help seeking and assistance at a productive granularity; iSnap and LLM hint systems similarly expose next-step or multi-level support [Aleven and Koedinger 2000; Heickal and Lan 2024; Price et al. 2017; Roest et al. 2024; Xiao et al. 2024]. The ZPD motivates support between independent and assisted performance [Chounta et al. 2017; Cole et al. 1978; Murray and Arroyo 2002; Schulze Bernd and Chounta 2024]. We operationalize only observed reachability: a later improving submission proves that this user reached that program state, not that showing the edit causes learning or is optimal scaffolding.

Submission sequences also support knowledge tracing, which estimates latent mastery from interaction histories [Corbett and Anderson 1994; Ghosh et al. 2020; Piech et al. 2015a]. ZPDPatch instead orders source-code states using execution evidence; it neither infers mastery nor predicts a learner response. This distinction permits fully automatic repair targets while delimiting the contribution to program behavior rather than educational outcomes.

2.4 Positioning of This Work

Unlike reference alignment and LSGen, ZPDPatch does not require a same-problem solution repository; unlike prior LLM and iterative repair, its targets are mined as ordered transitions from the same user's authentic submissions. Its multi-checkpoint control must also be distinguished from deep ensembles and model soups, which aggregate predictive distributions or weights [Lakshminarayanan et al. 2017; Wortsman et al. 2022]. We instead execute complete programs, measure set-union repair coverage on validation problems, and retain separate checkpoints for conditional early stopping and structural gating. Because seed diversity alone can create complementary candidates, our evaluation gives a same-target Answer pool the identical number of trained checkpoints, exhaustive size-three choices, and test-time generation budget as the mixed-target pool.

Repeated sampling is an established inference-scaling mechanism for code: pass@k rises when independent draws cover different successful programs [Chen et al. 2021]. We do not claim that effect as novel. Instead, our same-draw T_1–S_3 construction measures its exact contribution inside an APR controller, then holds the three-call budget fixed while testing whether independent checkpoints or relation-mined targets add coverage or structural locality beyond sampling alone.

A direct numerical port of prior systems would either grant held-out tasks method-only evidence or remove a defining component; query budgets also differ. We instead use matched controls: iterative execution-feedback Zero-shot for adaptation, Answer for target/checkpoint diversity, and LSGen for same-problem retrieval under the common runner. Unlike adaptive PyDex, CREF, and counterexample-guided repair [Orvalho et al. 2025; Yang et al. 2024; Zhang et al. 2024], our candidates do not alter later queries; this makes their set-union coverage exactly replayable.

3 Problem Formulation

For problem p, let a student's i-th submission be $s_i = (c_i, v_i)$, where c_i is the complete source code and v_i is the online-judge verdict. A submission trajectory $\tau = [s_1, s_2, \ldots, s_n]$ is chronologically ordered and terminates at the first accepted submission. Let d_p contain the problem statement and resource constraints, and let $\mathcal{E}_p$ be the available test suite. Re-executing c_i gives a per-test outcome vector $o_i = (o_i(e))_{e \in \mathcal{E}_p}$.

A repair example consists of problem context d_p, an observed sequence h, and a complete target program y. Adapter-specific rules decide which submissions remain in h and which later submission supplies y; the two need not be adjacent in the original trajectory. Given $x = (d_p, h)$, a model generates one complete program $\hat{c}$.

We call a later state *reachable* when it occurs in the same student's observed trajectory and satisfies an adapter-specific improvement rule. Reachability is an empirical property of the data, not a guarantee that the transition is minimal, comprehensible, or caused by instructional support. Our design hypothesis is that learning from such transitions can produce repairs that preserve more of the current solution than direct answer generation.

4 Approach

4.1 Design Requirements

Four requirements shape the method: *automatic supervision* recoverable from submissions and tests; *target diversity* spanning intermediate improvement and accepted solutions; *executable validation* independent of model confidence; and *non-regression* when no candidate improves observed behavior. We separate these offline and online decisions. Label relations determine what each adapter learns; execution and selection determine what the system may return. A noisy target can influence generation without automatically overriding a better current program.

4.2 Trajectory Construction

We group submissions by problem and student, sort by time, and stop at first acceptance. After indentation/comment/blank-line normalization, we remove only consecutive duplicates; non-consecutive revisits remain evidence of distinct attempts. Complete source, verdict, and order are preserved, with no maximum history or transition count. Every state is re-executed. Cache keys bind problem, normalized code, test-suite identity, timeout, and memory limit, preventing incompatible reuse. Runner failures receive an explicit most-severe outcome, and Progress construction requires a complete cache.

4.3 Adapter-Specific Supervision

We order verdict severity as $\mathrm{sev}(\texttt{AC}) = 0$, $\mathrm{sev}(\texttt{WA}) = 1$, $\mathrm{sev}(\texttt{TLE/MLE}) = 2$, $\mathrm{sev}(\texttt{RE}) = 3$, $\mathrm{sev}(\texttt{CE}) = 4$, and 5 for internal failures. This is an explicit operational total preorder for mining labels, not a claim that, for example, every WA program is semantically closer to correctness than every RE

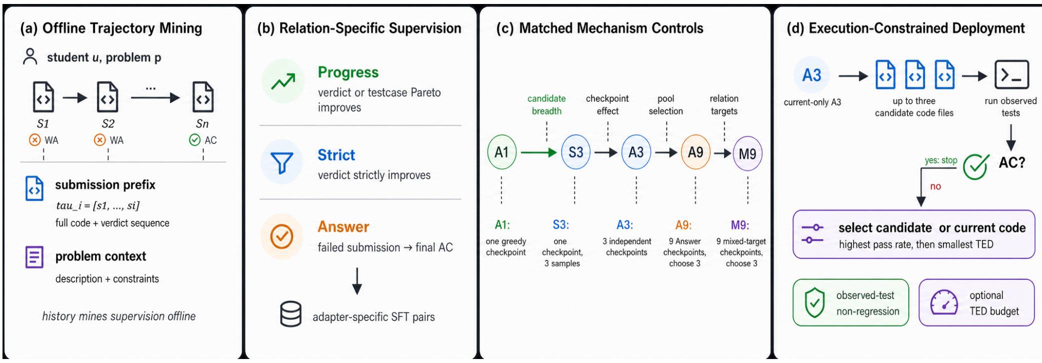

Fig. 1. Final identification and deployment design. Histories mine relation-specific SFT pairs offline. The matched $A_1 \to T_1 \to S_3 \to A_3 \to A_9 \to M_9$ ladder separates candidate breadth, checkpoint effects, pool selection, and relation targets. Recommended current-only A_3 generates up to three candidates; execution stops on AC and otherwise applies non-regressive, TED-aware selection.

program. For equal, non-empty test keys, $o_t \preceq_{\text{sev}} o_r$ iff no test outcome in o_t is more severe than its counterpart in o_r. Test-case progress is the Pareto condition

$$\text{TCProg}(r, t) = \mathbf{1}[o_t \preceq_{\text{sev}} o_r \wedge o_t \neq o_r] \,. \tag{1}$$

Answer. If s_n is accepted, every preceding non-accepted submission is independently paired with c_n. For $\tau = [S_1, \ldots, S_9]$, the rule produces $([S_1], c_9), \ldots, ([S_8], c_9)$. Thus, Answer learns a direct failed-program-to-answer mapping and receives no multi-submission history.

Strict. Starting with $R = [s_1]$, we scan the trajectory and append s_t only when its verdict is strictly better than the last retained submission $s_r = R[-1]$: $\text{sev}(v_t) < \text{sev}(v_r)$. If $R = [r_1, \ldots, r_m]$, we create $([r_1, \ldots, r_{k-1}], c_{r_k})$ for every $k = 2, \ldots, m$. The comparison is against the last retained state, not necessarily the immediately preceding original submission.

Progress. Progress uses the same retained scan but appends s_t when either its verdict strictly improves or $v_t = v_r$ and $\text{TCProg}(r, t) = 1$. It therefore captures monotonic per-test progress that is hidden by an unchanged coarse verdict. Prefix-target construction is otherwise identical to Strict.

The datasets are constructed independently and may overlap. A strict transition is also a progress transition, and final accepted transitions can occur in both. The objectives nevertheless differ: Answer learns direct completion from one failed program, whereas Strict and Progress learn transitions from retained prefixes.

The order is consequential and auditable. Of the retained training labels, 89.5% of Strict and 90.7% of Progress remain valid if RE is placed below WA; 82.6% and 84.6% remain valid when every non-AC verdict is tied. This retrospective label audit does not estimate retrained-model performance, so we treat the chosen order as part of the method specification rather than a universal correctness ranking.

4.4 Execution-Evidence Order

The three rules are not unrelated task labels. They form two nested execution-evidence orders plus a terminal closure. Let the evidence of submission (s) be (e(s)=(v(s),o(s))) over the fixed testcase universe $\mathcal{E}_p$. Define the strict relation

$$s \prec_S t \iff \text{sev}(v(t)) < \text{sev}(v(s)), \tag{2}$$

and the progress relation

$$s \prec_P t \iff s \prec_S t \;\vee\; (v(s) = v(t) \wedge o(t) \prec_\times o(s))\,, \tag{3}$$

where $o(t) \prec_\times o(s)$ is the strict product order: every testcase outcome is no worse and at least one is better under severity. Strict and Progress retain a chain under $\prec_S$ and $\prec_P$, respectively. Answer is the terminal closure $A(s) = s_n$ from each failed state to the first accepted state; it is intentionally not an intermediate order.

Strict is a coarse projection, Progress its non-compensatory testcase refinement, and Answer a closure to acceptance. For T tests, every transition decreases $\rho(s) = (\mathrm{sev}(v(s)), \sum_e \mathrm{sev}(o(s,e)))$ lexicographically, so every mined chain is acyclic. Moreover, $\prec_S \subseteq \prec_P$; these facts induce the fine-to-broad controller order.

Unlike a passed-test scalar, the product order cannot trade a newly broken test for another repaired test. Its guarantee is limited to supervision construction: learned candidates still require execution and the original fallback. The order is defined over one fixed testcase universe and severity map; cache keys bind both tests and resource limits, and Progress fails closed on incomplete vectors.

4.5 Prompt and Adapter Training

All adapters share one role-neutral prompt renderer: statement and limits, followed by every submission stored in that training record in chronological order, with verdict/outcome/edit summaries, and a request for one complete solution in the dataset's language and format. The construction rule therefore determines the training context: Answer records contain only the current failed submission, whereas Strict and Progress records contain the retained prefix through that submission. No instruction names a policy or asks for a particular edit size, so specialization can arise only from the input–target distribution. Only assistant target tokens contribute to loss.

Canonical validation and test use one common prompt distribution: every checkpoint, including Answer, receives the complete authentic prefix available at the query. Answer therefore has a deliberate train–inference context shift, not privileged input. RQ6 separately replays the frozen full-prefix members and reselects both nine-checkpoint pools using current-code-only validation prompts, so prompt distribution and target composition are not inferred from one contrast.

Each adapter independently minimizes mean target-token cross entropy with the base weights frozen and its own QLoRA parameters; prompt tokens are masked.

4.6 Validation-Selected Execution Portfolio

Training loss ranks checkpoints within one target distribution, but it does not measure the cross-policy repair complementarity used at deployment. We therefore select the portfolio using execution on validation problems. Let $V = V_P \,\dot\cup\, V_S \,\dot\cup\, V_A$ be independently trained Progress, Strict, and Answer checkpoints and let $R_v \subseteq U$ be the validation instances repaired by checkpoint v. Portfolio coverage is

$$f(S) = |\bigcup_{v \in S} R_v| / |U|. \tag{4}$$

We search $\mathcal{F}_3 = \{S \subseteq V : |S| = 3\}$, allowing validation to choose relations and seeds, and retain $\mathcal{F}_{\mathrm{rel}} = \{S \in \mathcal{F}_3 : |S \cap V_a| = 1\ \forall a\}$ as the direct relation-heterogeneity control. We sample one eligible trajectory per validation problem by a fixed hash before any execution, yielding 461 problems with complete nine-checkpoint evaluation. For predeclared $\mathcal{B} = \{5, 10, 20, 40, 80, 160\}$, let R_v^B retain repairs with AST TED at most B, define $f_B(S) = |\bigcup_{v \in S} R_v^B| / |U|$, and use $f_{\mathcal{B}}(S) = |\mathcal{B}|^{-1} \sum_{B \in \mathcal{B}} f_B(S)$ when one triple must span budgets. A budget known before a query instead uses the validation maximizer of f_B. We enumerate all $\binom{9}{3} = 84$ unrestricted and 27 relation-constrained triples for every objective; a fixed three-Answer portfolio is the same-target control. Test outcomes never enter selection.

These set-union objectives are monotone. Because the family contains only 84 triples, enumeration returns the exact empirical optimum; five-fold problem-cross-fitting, rather than a loose distribution-free finite-family bound, estimates its held-out behavior.

Mechanism-identification ladder and exact breadth decomposition. Let $r(P)$ be held-out RR for controller P, A_1 one greedy Answer checkpoint, T_1 one uniformly chosen member of three fixed stochastic draws, S_3 the execution-selected union of those *same* draws, A_3 the fixed three-seed greedy Answer controller, A_9 Answer-9Choose3, and M_9 mixed-target-9Choose3. The planned contrasts

$$r(T_1) - r(A_1), \quad r(S_3) - r(T_1), \quad r(A_3) - r(S_3), \quad r(A_9) - r(A_3), \quad r(M_9) - r(A_9) \tag{5}$$

separate decoding stochasticity, same-draw candidate breadth, an aggregate checkpoint/decoding contrast, added pool/selection opportunity, and relation-specific dataset construction. Because S_3 is stochastic whereas A_3 is greedy, $A_3 - S_3$ alone does *not* identify checkpoint diversity; RQ4 therefore adds a decoding-matched three-checkpoint stochastic cell and a five-temperature sweep. For fixed candidates, let Z_{ij} indicate that draw j repairs instance i, and $M_i = \sum_{j=1}^{k} Z_{ij}$. The realized breadth contribution is exactly

$$\widehat{\Delta}_{\text{breadth}} = \frac{1}{N} \sum_{i} \left[\mathbf{1}(M_i > 0) - \frac{M_i}{k} \right] \geq 0. \tag{6}$$

It is positive only when candidates disagree in success; it is zero when all fail or all succeed. Thus Equation 6, estimated by the three single-draw outcomes and their identical union, attributes no gain to a temperature change. Under conditionally independent draws with success probability p_i, its dataset-level expectation is $N^{-1} \sum_i [1-(1-p_i)^k - p_i]$, connecting execution selection to standard pass@k/best-of-n inference scaling [Chen et al. 2021] rather than treating repeated sampling as a new training effect. Only the last ladder contrast matches pool size, 84-way search, validation data, selector, and three-call budget. Answer also has more training examples, so it is a compute-favored control, but the target-distance distributions also differ. The last contrast therefore estimates the complete mixed-target construction conditional on the declared seed family, not relation semantics holding every training-distribution property fixed.

4.7 Execution-Guided Sequential Repair

At inference, we invoke the three validation-selected checkpoints in fine-to-broad relation order, breaking same-relation ties by their frozen member order. Each checkpoint receives the same complete observed evaluation trajectory, including Answer checkpoints whose SFT records contained only one submission, and produces one complete program. This fixes available evidence across policies without claiming that Answer was history-trained. Candidates never enter later prompts, avoiding synthetic off-distribution trajectories. Unrestricted execution stops at the first acceptance; budgeted execution continues past over-budget acceptances. The order is frozen before evaluation, and generated-feedback prompting is evaluated separately.

If no eligible candidate is accepted, the selector includes the current program c_i as a fallback:

$$C = \{c_i, c^{(1)}, c^{(2)}, c^{(3)}\}. \tag{7}$$

Let $\text{Pass}(c)$ be the fraction of tests passed. We define $\text{ted}^{\dagger}(c_i, c) = 0$ for the unchanged fallback, the Python AST tree edit distance when both changed programs parse, and $+\infty$ otherwise. Fallback selection is lexicographic:

$$c^* = \arg\max_{c \in C} \left(\mathrm{Pass}(c), -\mathrm{ted}^\dagger(c_i, c) \right). \tag{8}$$

The fallback selector maximizes executed correctness, then minimizes change; when no candidate improves c_i, the zero-TED fallback makes the system abstain.

Certified selection and cost.

Theorem 1 (Controller contract). *For any non-accepted current program, ordered list of generated candidates, and budget B, the controller (i) returns a program whose observed pass rate is at least that of c_i, (ii) returns a changed program only if its finite AST TED is at most B, and (iii) returns an accepted program iff at least one emitted candidate is both accepted and budget eligible. Consequently, member order can change calls and which accepted patch is returned, but not repair coverage for a fixed candidate set.*

Proof. An eligible accepted candidate terminates the scan and has pass rate one. If none exists, Equation 8 maximizes over a set containing the zero-edit fallback, proving (i); the eligibility filter proves (ii). The scan visits every candidate until an eligible acceptance, proving both directions of (iii). The final statement follows because existence in a finite set is permutation invariant. □

Thus deployed coverage is exactly the validation set-union objective f_B. The contract concerns observed tests and edit compliance, not hidden-test correctness or learned relational behavior.

Construction costs $O(nT)$ time and $O(n)$ storage; Answer is $O(n)$. The $A_1/A_3/A_9/M_9$ ladder trains 1/3/9/9 adapters for 5.84/17.5/52.6/37.1 RTX 5090 GPU-hours and uses 0/0/4,149/4,149 portfolio-selection validation executions. Its Seen RR is 50.1/61.7/59.8/61.5%, with 1.00/1.93/1.93/1.98 mean deployment calls. Thus A_3 dominates A_9 in unrestricted RR and cost; the nine-model pools are diagnostic and support the edit-constrained frontier, not a universally cheaper default. M_9/A_3 latency is 13.62/13.32 seconds versus 24.40 for always-three LSGen, excluding cached current execution. Nine 155MB LoRA files occupy 1.4GB; deployment loads three.

5 Experimental Design

5.1 Research Questions

- **RQ1: Utility and Frontier.** What unrestricted and edit-constrained utility does the execution-controlled portfolio provide relative to zero-shot and retrieval-based repair?
- **RQ2: Trajectory Conditioning.** Does access to the full submission prefix improve repair over training and inference with only the current code?
- **RQ3: Adapter Specialization.** Do Progress, Strict, and Answer exhibit different repair coverage and patch size?
- **RQ4: Breadth Identification.** How much coverage comes from output sampling, independently trained checkpoints, validation pool breadth, and relation-specific dataset construction?
- **RQ5: Problem Generalization.** Does ZPDPatch retain an advantage on problems completely excluded from training?
- **RQ6: Robustness and Replication.** Are conclusions robust to scale, seeds, prompt and verdict choices, dependence, ordering, and the Java replications?

"Frozen" excludes final-test outcomes from training and selection. Canonical tests, the ladder, RQ2, and student-heldout Java were selection-frozen; hidden/scale/language, prompt, sampling, cross-fit, and verdict-order controls were repository-frozen before their outcomes; normalized TED, retention, user strata, order replay, and heterogeneity are exploratory. None was externally preregistered.

During scale execution, an invariant exposed repeated current-program execution that violated the already frozen cache rule. Before portfolio selection or analysis, a logged conformance amendment rebound every member to the pre-existing cached baseline; it changed no training, generation, generated-program outcome, TED, selector, or analysis code. Downstream runners were corrected before execution. Conformance audits are specification checks, not effect tests.

RR is the primary effectiveness endpoint. The system contrast is portfolio versus Zero-shot; the mechanism contrast is selection-matched $M_9 - A_9$. For the post-review family, pooled five-fold problem-cross-fit Seen $M_9 - A_9$ is the primary mechanism estimate and its mean over the six fixed TED budgets is secondary. Scale and Java replications repeat those endpoints; prompt and verdict-order reruns are sensitivity analyses. Individual budgets and heterogeneity strata remain descriptive and do not select the conclusion.

5.2 Dataset Construction and Splits

We combine Python submissions, statements, metadata, and test cases for problems in Project CodeNet's Python800 benchmark [Puri et al. 2021]. We retain trajectories with at least three submissions, at least one non-accepted state before the first acceptance, and a final accepted program present in the Python800 reference set. We remove post-acceptance submissions and consecutive normalized duplicates. Users must appear in at least three problems and each retained problem must contain at least two trajectories. The refined corpus contains 546 problems, 21,454 trajectories, 99,432 submissions, and 10,088 tests.

CodeNet identifies online-judge users and attempts, not classroom enrollment; we therefore use it as behavioral repair evidence rather than evidence about students or learning. The separately collected CodeWorkout replication below provides the CS1-course population.

Before splitting, we tokenize every possible Answer, Strict, conservative Progress, and final-evaluation prompt-target configuration. If any configuration exceeds 4,096 tokens, we exclude the entire trajectory rather than truncating code or history. This removes 2,896 trajectories and leaves 18,558. No later `max_history`, transition-count, code, or test truncation is applied.

We sort problems by eligible trajectory count and assign the top 90% (492 problems) to *Seen* and the remaining 54 low-resource problems to *Unseen*. Within every Seen problem, we reserve at least one trajectory for Train, Validation, and Test, then obtain a deterministic 80:10:10 trajectory split. All 260 Unseen trajectories remain problem-held-out test data. Table 1 reports the resulting corpus. Users may occur in more than one split; RQ5 is problem-held-out, not user-held-out.

Adapter construction yields the training and validation sizes in Table 2. Final evaluation uses the prefix ending at the last non-accepted submission and the final accepted program as the oracle. We retain only cases for which re-execution confirms the current program as non-accepted and the oracle as accepted, leaving 997 Seen instances from 328 of the 492 Seen problems and 250 Unseen instances from all 54 held-out problems. The remaining Seen problems still contribute Train or Validation trajectories but have no Test trajectory satisfying both execution checks; this is why the abstract's test-problem count is smaller than the Seen pool.

To test the educational and language scope directly, we add TIKTOC's testcase-augmented CodeWorkout release, which contains authentic CS1 Java submissions and per-test outcomes [Duan et al. 2025]. We apply the same first-acceptance, duplicate, minimum-length, and 4,096-token whole-trajectory rules, then split by student rather than trajectory. The resulting external corpus has 1,125 trajectories from 224 students over all 17 exercises, with zero student overlap: 786/158/181 train/validation/test trajectories from 157/34/33 students. Every split contains all 17 exercises. External training contains 3,470 Answer, 1,110 Strict, and 1,386 Progress examples; validation contains 730/230/276. We train the same model and hyperparameters from the same base checkpoint. Seeds 2027/2028/2029 per relation create the mixed-target pool; Answer seeds 2030–2035 complete a

Table 1. Dataset statistics after whole-trajectory context filtering. “Prefix” counts possible raw prefixes before adapter-specific filtering.

Group	Role	Problems	Tests	Traj.	Sub.	Prefix	Users
	Train			14,638	58,872	44,234	4,072
Seen	Validation	492	9,446	1,830	7,283	5,453	1,283
	Test			1,830	7,068	5,238	1,291
Unseen	Test	54	642	260	965	705	237

Table 2. Adapter supervision datasets. Validation examples are excluded from training.

Adapter	Train examples	Validation examples
Progress	21,416	2,685
Strict	16,973	2,116
Answer	40,454	4,965

selection-matched nine-Answer pool. We execute both pools on the 158 student-disjoint validation states, enumerate all 84 size-three portfolios per pool plus 27 one-per-relation portfolios, and freeze every winner before testing. The 181 student-held-out final states compare M_9, A_9, and the fixed A_3; no CodeWorkout test outcome enters checkpoint or portfolio selection.

5.3 Baselines

Zero-shot. Despite its name, Zero-shot is an execution-aware iterative baseline: it uses the same Qwen base model and prompt as ZPDPatch without fine-tuning, and after a failure adds the verdict and number of passed tests before its next call. It receives at most three generations, matching ZPDPatch’s maximum budget. Because it receives the full trajectory, the contrast measures the complete effect of learned adapters and sequential composition, not history in isolation. For the decoding-matched breadth audit, an additional *Base*×3 control samples the unadapted Qwen checkpoint three times from the identical current-only prompt and with the identical seeds, temperature, top-p, token limit, and execution selector as Answer×3. This isolates whether fine-tuning adds coverage beyond stochastic breadth in the base model.

Supervised Answer ensemble. Answer-3Seed is an inference-matched learned baseline: three conventional current-code-to-accepted-answer SFT adapters use the same 40,454 examples, base model, hyperparameters, greedy decoding, and maximum three-generation budget as ZPDPatch, differing only in training seed. Answer-9Choose3 is the selection-matched control: nine independently seeded Answer adapters receive the identical exhaustive $\binom{9}{3} = 84$-portfolio validation search as the nine-checkpoint mixed-target pool. Their SFT records contain no earlier submissions, but at validation and test time they receive the same complete evaluation trajectory as every mixed-pool checkpoint. Thus prompt information, decoding, and portfolio opportunity are matched; the compared difference is the learned checkpoint pool induced by the training relations. Answer-9Choose3, rather than Zero-shot or Answer-3Seed, isolates whether mixed trajectory targets add coverage beyond ordinary checkpoint diversity and validation selection opportunity.

LSGen. LSGen is an iterative retrieval-based solution generator [Dai et al. 2026]. We preserve its repair-pair filtering, edit-vector retrieval, diff-guided generation, and re-retrieval, while replacing hard-coded infrastructure with our common dataset and runner. For a Seen query, the retrieval database contains buggy/correct pairs from other Seen-Train trajectories of the same problem; the same student and example are excluded. LSGen retrieves five pairs and performs at most three iterations. We do not evaluate it on Unseen problems because exposing same-problem correct programs would violate problem holdout.

All approaches use the same base LLM, public tests, 2.5-second per-test timeout, 2GB memory limit, and a maximum of three candidate generations. Final validation-selected, external-replication, and budget-controller runs cap every new candidate at 4,096 new tokens; reused control candidates are audited against the same bound.

5.4 Implementation

We use Qwen2.5-Coder-7B-Instruct [Hui et al. 2024] and train each adapter for one epoch with 4-bit QLoRA [Dettmers et al. 2023]. The base weights use NF4 double quantization and BF16 computation. LoRA targets the q, k, v, o attention projections and gate/up/down MLP projections with rank 16, scaling 32, and dropout 0.05. Optimization uses paged 8-bit AdamW, learning rate 2×10^{-4}, cosine scheduling, 0.03 warmup ratio, and gradient checkpointing. Both canonical and CodeWorkout training use micro-batches of 2 with eight accumulation steps, for an effective batch of 16. Seeds 2027/2028/2029 repeat every target; Answer-9Choose3 adds Answer seeds 2030–2035 without changing data, hyperparameters, or greedy decoding. We select the lowest validation-loss checkpoint and run on one NVIDIA RTX 5090.

Two replications retain the same targets, selectors, and effective batch. The scale replication substitutes Qwen2.5-Coder-1.5B-Instruct and uses micro-batches of 4 with four accumulation steps. To avoid duplicating its large-vocabulary logits, the algebraically identical weighted FP32 token loss is reduced one sequence at a time with backward recomputation. The assignment-held-out CodeWorkout split uses 10/3/4 disjoint train/validation/test exercises (605/216/304 trajectories) and retrains both nine-checkpoint pools. It contains 213/150/161 students, including 151 shared train–test identities; it therefore isolates assignment transfer, whereas the student split isolates learners.

OpenAI Codex was used interactively to propose control and validation-selection design alternatives, implement dataset builders, orchestrate experiments, develop statistical-analysis scripts, and assist manuscript drafting. The authors selected the final design and acceptance criteria, executed every job on author-controlled infrastructure, reviewed code changes, and checked reported values against saved machine-readable outputs. AI-produced text was not treated as empirical evidence, and every cited work was checked against a retrievable source.

An isolated runner executes original and generated programs: Python800 uses the Python judge and CodeWorkout compiles Java 21 in a network-disabled container. We cache outcomes by problem, code, test set, timeout, and memory configuration and reuse the cache across supervision construction and evaluation. Within an evaluation split, every policy is bound to the same cached current-program verdict and pass rate; only a previously unseen generated candidate is executed, which prevents timeout-boundary noise from changing the baseline across portfolio members. The Java harness reproduces all 181 recorded accepted test programs before model evaluation.

5.5 Ablations

For RQ2, we construct adapter-specific Strict and Progress examples for both Seen and Unseen tests. *Full Trajectory* uses each retained prefix. *Current Code Only* retrains an adapter on identical examples and targets after removing every submission before the current one; its displayed position is reset to S_1 to prevent index leakage. Answer is excluded because its inputs already contain one submission. This design isolates the presence of history while holding targets, counts, model, and hyperparameters fixed.

For RQ3, each adapter independently generates one candidate for the same 997 Seen instances. For RQ4, we first compose the seed-2027 candidates with Progress–Strict–Answer early stopping and fallback selection, then evaluate the final validation-selected size-three portfolio. Five controls separate target heterogeneity from repeated calls, prompt-state effects, training randomness, pool size, and the relation constraint. *Answer*×3 invokes the same Answer adapter up to three times with generated execution feedback; repeating the identical prompt would collapse to Answer once. *Generated Feedback* applies that update protocol to heterogeneous P–S–A. *Answer-3Seed* uses identical Answer data and targets with seeds 2027/2028/2029. *Answer-9Choose3* uses seeds 2027–2035 and the same 461-problem validation executions, exhaustive 84 triples, objectives, tie breaking, and frozen testing as the mixed-target pool. *Validation Relation-Constrained* selects one checkpoint per relation; the primary *Validation Execution Portfolio* selects any three of the nine mixed-target checkpoints. Every member receives the same complete authentic evaluation trajectory without generated feedback, even though Answer SFT used single-submission records, and all configurations share a maximum three-generation budget, execution limits, early stopping, and fallback selector. We audit matching Answer metadata and distinct adapter hashes. Effectiveness-only controls may omit AST TED when failed pass-rate ties use the earliest candidate; Answer ensembles and validation-selected portfolios use the complete production selector.

Four repository-frozen controls address remaining identification choices. The *prompt control* removes prior submissions and reports both reselected and frozen-member contrasts. The *problem cross-fit* hashes Seen problems into five folds and excludes each test fold's identities during selection. The *verdict-order control* collapses all non-AC verdicts, rematerializes Progress/Strict pairs, and retrains both 7B adapters from the same base and seed. The *same-draw sampling control* draws three non-adaptive candidates from one Answer checkpoint at temperature 0.8/top-p 0.95, evaluates each as a one-candidate T_1 replicate and their identical union as S_3, and then compares greedy A_1 and A_3. Endpoints, folds, selectors, budgets, and 10,000 problem-bootstrap draws were fixed before test generation; none is externally registered. To remove the remaining checkpoint/decoding confound, a second frozen audit uses the same current-only prompt, seeds 4101–4103, and stochastic decoder for three cells: three draws from Answer2027; one draw from each of Answer2027/2028/2029; and three draws from the untuned base model. It also sweeps the one-checkpoint union over temperatures 0.2, 0.4, 0.6, 0.8, and 1.0 at fixed top-p = 0.95. We report every temperature on both splits; test results do not select a temperature or portfolio.

Independent-test confirmation. To separate candidate selection from success assessment, post-review audits partition each problem's tests by fixed SHA-256 hashes before regeneration. For Unseen (seed 2027), the observed side contains 332 tests and the hidden side 310, with at least three of each for all 54 problems. We recompute every displayed verdict, pass rate, and failure signature from observed tests only, regenerate the frozen ZPDPatch and Answer-9Choose3 members, and select candidates using only observed execution. A *joint repair* must pass both observed and untouched hidden tests; hidden outcomes never enter the prompt, generation, early stopping, tie breaking, or method selection. We repeat the procedure for all 328 Seen problems with a distinct seed (52027), where same-problem training solutions make leakage risk greatest.

5.6 Metrics and Statistical Analysis

For M instances, let c_m be the current program and $\hat{c}_m$ the selected result. $\mathrm{Pass}(c) \in [0, 1]$ is the test pass fraction. Pass Rate (PR) averages $\mathrm{Pass}(\hat{c}_m)$; Repair Rate (RR) averages $\mathbf{1}[\mathrm{Pass}(\hat{c}_m) = 1]$; Improvement Rate (IR) averages $\mathbf{1}[\mathrm{Pass}(\hat{c}_m) > \mathrm{Pass}(c_m)]$.

For successfully repaired, parseable programs, $\mathrm{TED}_{B\to F}$ measures AST distance from current to fixed code and $\mathrm{TED}_{F\to O}$ from fixed code to the recorded accepted oracle, using the standard unit-cost ordered-tree edit formulation [Zhang and Shasha 1989]. Legacy fixed-policy reports use ZSS; validation-selected portfolios and exhaustive budget replay use the equivalent APTED implementation [Pawlik and Augsten 2016], cross-checked against ZSS on identical labeled ASTs. Because node-count difference lower-bounds unit-cost TED, pairs whose difference already exceeds the largest declared budget are stored only as > 160, which preserves every budget decision. RQ2 uses $\mathrm{TED}_{F\to T}$, where T is the identical adapter-specific recorded target shared by Full and Current configurations.

We report Wilson 95% intervals for RR and IR [Wilson 1927]. Paired RR comparisons use two-sided exact McNemar tests [McNemar 1947], with Holm correction within each planned comparison family [Holm 1979]. Because trajectories from one problem are dependent, our primary robustness analysis resamples problems as clusters 10,000 times with seed 2027 and reports intervals for paired RR, PR, and IR differences. We additionally report a problem-balanced estimand that gives every problem equal weight, with a problem bootstrap interval. CodeWorkout contrasts report both exercise- and student-cluster intervals because either identity can repeat within its held-out split. Paired TED differences use 10,000 instance bootstrap resamples on jointly repaired inputs; both procedures follow the nonparametric bootstrap principle [Efron 1979]. Unless stated otherwise, percentages are absolute percentage points.

6 Results

6.1 RQ1: Repair Effectiveness

Table 4 summarizes RQ1 and RQ5. On Seen problems, the strongest learned configuration is current-only Answer-3Seed: it repairs 72.0% Seen and 81.2% Unseen, versus 61.5% and 74.8% for the full-history mixed-target portfolio. We therefore expose it as the unrestricted deployment default rather than treating it as a secondary ablation. The mixed pool remains the test vehicle for relation-mined supervision and the small-budget frontier.

On Seen problems, the validation-selected mixed execution portfolio repairs 613 of 997 programs (61.5%), compared with 306 (30.7%) for Zero-shot. Their Wilson RR intervals are [58.4, 64.5] and [27.9, 33.6], respectively. ZPDPatch alone repairs 342 instances, whereas Zero-shot alone repairs 35 (exact McNemar $p < 10^{-63}$). More importantly for dependent trajectories, the 30.8-point paired RR gain remains under problem-cluster bootstrap [26.4, 35.1]; the equal-problem-weight gain is 26.1 points [21.2, 30.9]. PR increases by 11.07 points [9.16, 13.02] and IR by 25.2 points [20.8, 29.4] under the same clustered analysis.

The selection-matched Answer-9Choose3 baseline is the strictest target comparison: ZPDPatch differs by +1.71 Seen RR points [−0.79, 4.29] and 0.0 Unseen points [−3.70, 3.75]. Thus unrestricted results establish portfolio repair utility, but not a general coverage gain from relation-mined targets over an equally large pool of independently trained Answer checkpoints.

The legacy LSGen controller achieves the highest Seen execution correctness: 88.0% PR, 80.2% RR, and 82.4% IR. It alone repairs 267 instances, while ZPDPatch alone repairs 80 (exact McNemar $p < 10^{-23}$); the clustered RR difference is −18.8 points [−23.0,−14.5]. The tradeoff is patch size. ZPDPatch's mean $\mathrm{TED}_{B\to F}$ is 21.05, versus 27.25 for Zero-shot and 88.27 for LSGen. Among jointly repaired parseable instances, ZPDPatch reduces TED by 8.46 relative to Zero-shot (95% CI [5.05, 12.55]) and by 43.34 relative to LSGen (95% CI [38.47, 48.44]). RQ6 replaces the legacy early-stop replay with an always-three LSGen controller before making the final budget-frontier comparison.

Table 3 operationalizes this tradeoff rather than treating TED as evidence of pedagogical quality. Under a maximum structural edit budget, ZPDPatch has substantially higher repair coverage through $B = 80$; LSGen crosses over only at the broadest reported budget. At $B = 20$, for example, the gate returns an accepted repair for 44.0% of inputs with ZPDPatch versus 13.9% with always-three LSGen, a +30.09-point problem-clustered difference [26.45, 33.82]. Across all six predeclared budgets, the mean difference is +14.68 points [12.03, 17.43]. The comparison therefore supports a constrained deployment claim: retrieval has higher unrestricted coverage, whereas the trajectory-trained portfolio covers more inputs when large structural replacement is disallowed. We treat TED only as an explicit change-cost axis: the benchmark supplies no evidence that a particular budget is an educational requirement or that a smaller patch improves learning.

Paired outcomes show complementary success rather than marginal exchanges. On Seen, ZPDPatch and Zero-shot repair 271 jointly, with 342 versus 35 unique repairs; against LSGen the corresponding counts are 533 jointly and 80 versus 267 uniquely. On Unseen, ZPDPatch and Zero-shot repair 142 jointly, with 45 versus 13 unique repairs. Thus LSGen has higher coverage, but neither Seen success set contains the other.

PR and IR confirm that the RR gain is accompanied by partial behavioral improvement. Patch distance separates repair from replacement: mean current-to-fix TED is 22.8% below Zero-shot and 76.2% below legacy LSGen on each method's successful set, while the jointly repaired paired analyses above establish the same direction without success-set confounding.

6.2 RQ2: Trajectory Conditioning

Table 5 shows no consistent advantage for Full Trajectory. On Seen Strict examples, Full raises PR by 0.7 points but lowers RR and IR by 0.7 and 1.3. On Seen Progress, it lowers PR, RR, and IR by 1.0, 0.4, and 0.6. On Unseen data, Full is worse for Strict but better for Progress. Target TED is also mixed: Full is closer to the recorded target for Seen Progress and Unseen Strict, but farther in the other conditions. None of the four paired RR differences is significant after Holm correction (minimum adjusted $p = 0.313$).

Each Full–Current pair holds training examples, targets, architecture, hyperparameters, and evaluation inputs fixed; only earlier submissions are removed and position leakage reset. Direction changes across conditions, clustered RR intervals include zero (Unseen Strict reaches zero at its upper endpoint), and target TED is mixed. RQ1 can therefore establish complete-system utility while RQ2 does not support history serialization as a stable causal explanation.

We also searched directly for cases in which history should be most identifiable. A mechanically specified matched audit requires different users on the same problem, identical current verdict and pass rate, at least one earlier submission each, and normalized current-code similarity of at least 0.8. Only 6 Strict and 12 Progress Seen examples qualify (none Unseen); Full reaches 16.7/58.3% RR versus 66.7/66.7% for Current. These tiny, post-hoc subsets cannot estimate a population effect, but they expose rather than conceal that the benchmark contains little counterfactual support for a history-sensitive claim.

6.3 RQ3: Adapter Specialization

Table 6 exposes a coverage-size tradeoff. Answer has the highest PR, RR, and IR, but also the largest mean patch at TED 20.22. Progress has lower coverage but the smallest patch at TED 15.85. Strict and Progress have statistically indistinguishable RR (adjusted $p = 0.857$). Answer repairs 5.2 points more than Progress and 5.5 points more than Strict; problem-cluster intervals are [2.5, 7.9] and [2.9, 8.0], respectively.

The raw TED ordering could reflect different repaired subsets, so we also pair joint repairs and resample problems. On 350 Answer–Progress joint repairs, Progress is 1.84 TED smaller [0.50, 3.26]; on 361 Strict–Progress joint repairs, it is 1.67 smaller [0.53, 2.93]. Strict differs from Answer by only 0.29 [−1.09, 1.64] across 358 joint repairs. Thus the supported specialization is Progress's narrower patch scope, not a fully ordered three-policy hierarchy. Two source-based measures corroborate

Table 3. Seen repair rate (%) after the same AST-TED deployment gate. ZPDPatch uses the validation-frozen budget-indexed portfolio; current-only A_3 uses the recommended three checkpoints; LSGen always generates three candidates. The last column is ZPDPatch minus LSGen with a 95% problem-cluster interval.

Budget B	ZPDPatch	Current A_3	LSGen	ZPDPatch−LSGen [95% CI]
5	22.5	16.9	4.8	+17.7 [14.8, 20.6]
10	32.4	28.5	8.0	+24.4 [21.2, 27.7]
20	44.0	40.3	13.9	+30.1 [26.4, 33.8]
40	50.9	51.4	30.4	+20.5 [17.0, 24.0]
80	56.0	61.0	50.8	+5.2 [1.3, 9.3]
160	57.9	66.8	67.6	−9.7 [−14.2, −5.4]

Table 4. Main repair results. PR, RR, and IR are percentages; TED is computed on successfully repaired, parseable programs.

Split	Method	PR	RR	IR	$\text{TED}_{B\to F}$	$\text{TED}_{F\to O}$
Seen	Zero-shot	76.3	30.7	43.7	27.25	27.67
	LSGen	88.0	**80.2**	**82.4**	88.27	83.23
	Answer-3Seed (history)	87.0	61.7	69.1	24.52	24.82
	Answer-3Seed (current)	**90.2**	72.0	78.6	37.94	35.51
	Answer-9Choose3	85.9	59.8	68.0	25.46	24.01
	ZPDPatch	87.4	61.5	68.9	**21.05**	**21.64**
Unseen	Zero-shot	75.3	62.0	68.8	17.46	15.80
	Answer-3Seed (history)	84.8	73.2	79.2	13.38	13.86
	Answer-3Seed (current)	**89.3**	**81.2**	**88.0**	16.01	16.58
	Answer-9Choose3	85.7	74.8	80.4	15.19	13.75
	ZPDPatch	84.9	74.8	79.6	**11.40**	**11.96**

Table 5. RQ2 trajectory-conditioning results. PR, RR, and IR are percentages. Current and Full use identical examples and targets.

Split	Adapter	Input	N	PR	RR	IR	$\text{TED}_{F\to T}$
Seen	Strict	Current	2,151	56.8	31.2	54.6	40.23
	Strict	Full	2,151	57.5	30.5	53.3	41.05
	Progress	Current	2,674	58.4	26.3	49.7	36.23
	Progress	Full	2,674	57.4	25.9	49.1	33.15
Unseen	Strict	Current	322	66.7	56.2	71.4	20.20
	Strict	Full	322	65.4	53.4	70.8	19.27
	Progress	Current	378	65.7	52.1	66.4	17.63
	Progress	Full	378	66.5	53.7	68.3	18.16

rather than merely rename TED. On joint Progress–Answer repairs, Progress retains 1.29 more percentage points of buggy tokens [0.02, 2.57] and 2.89 more points of non-empty source lines [1.16, 4.71]. Progress also retains 0.84 more token points than Strict [0.04, 1.67], while the line interval crosses zero. The paired 1.84-TED reduction is about 9.1% of Answer's unpaired mean TED, whereas the 1.29-point token effect is modest; statistical detectability should not be confused with a

Table 6. RQ3 individual-adapter results on 997 Seen instances.

Adapter	PR	RR	IR	$\text{TED}_{B\to F}$	$\text{TED}_{F\to O}$
Progress	73.4	44.8	52.0	**15.85**	**15.89**
Strict	74.8	44.5	51.2	17.45	18.38
Answer	**77.5**	**50.1**	**57.5**	20.22	20.61

large practical change. These are preservation measures, not learner outcomes; hard non-regression enters only through selection.

6.4 RQ4: Mechanism Identification

The original Progress–Strict–Answer controller reaches 59.5% RR, 9.4 points [7.6, 11.3] above the strongest member, but does not identify why. Its early stop uses 2.04 calls per input instead of three, and fallback abstains 329 times. The complete ladder below separates the sources of its coverage.

Answer to RQ4. The complete ladder separates five mechanisms. Candidate breadth is the dominant unrestricted-coverage effect. Changing greedy decoding to one stochastic draw contributes -7.6 points [-9.81, -5.41], whereas execution-selecting three of those identical fixed draws contributes 15.1 points [13.48, 16.76]. Independent checkpoints then contribute 4.1 points [1.59, 6.79]. Enlarging only the Answer pool does not improve it: $A_9 - A_3 = -1.91$ [−4.30, 0.32]. The primary five-fold, instance-weighted $M_9 - A_9$ estimate is +2.2 [−0.19, 4.63], hence null; the non-cross-fitted estimate is +1.71 [−0.79, 4.29]. Equal-problem estimates are positive sensitivity analyses, not the confirmatory result. Thus mixed targets do not establish a robust unrestricted RR advantage. Repeating one checkpoint with generated feedback reaches only 55.2%, and compulsory one-per-relation selection reduces RR to 57.3%.

Relation targets instead produce a constrained-repair effect. Across six predeclared TED budgets, budget-specific M_9 portfolios improve mean Seen RR over selection-matched A_9 by 1.22 points [0.02, 2.42], with TED 20 at +1.61 [0.39, 2.85]. On 532 jointly repaired inputs, M_9 preserves 2.57 more buggy-token points [1.56, 3.64] and 3.33 more non-empty-line points [1.89, 4.83]. The method contribution is therefore target-dependent patch scope under an executable deployment constraint, while candidate breadth explains most unrestricted coverage.

A post-hoc normalized audit on 966 parseable inputs finds positive $M_9 - A_9$ effects at TED/current-AST limits 0.10, 0.20, and 0.40 (+2.80, +3.83, and +2.90 points), while 0.05, 0.80, and 1.60 include zero. This exploratory result neither replaces the fixed absolute frontier nor supports learner quality.

6.5 RQ5: Problem Generalization

On 250 Unseen instances, ZPDPatch reaches 74.8% RR versus 62.0% for Zero-shot: +12.8 points [6.83, 18.75], with 45 versus 13 exclusive repairs ($p = 3.01 \times 10^{-5}$). On 122 joint parseable repairs, its patch is 6.41 TED units smaller [3.43, 9.89]. Current-only A_3, however, is the strongest learned configuration at 81.2% RR (Table 4).

On Unseen, Independent Policies exceeds repeated Answer-with-feedback, 73.2% versus 68.4% RR (15 versus three exclusive repairs; McNemar $p = .00754$), but matches independent Answer-3Seed: 73.2%, difference 0.0 [−3.2, 3.1], $p = 1.0$. Thus problem-held-out evidence supports checkpoint diversity, not relation heterogeneity.

The frozen unrestricted M_9 and selection-matched A_9 winners both reach 74.8% Unseen RR. Their paired difference is 0.0 points [−3.70, 3.75], so enlarging the candidate pool does not reveal a mixed-target coverage effect. Budget-indexed $M_9 - A_9$ averages +1.40 points [−1.24, 4.02] across the

Illustrative trajectory (both observed attempts pass 93.3%): the user changes the outer bound from `i < j` to `i <= j`, but the right scan still uses `while j > 0`.

Progress repair (AC, TED 1): `while j >= 0` — a one-node boundary correction.

Answer repair (AC, TED 83): `cnt = S.count('R'); ans = S[:cnt].count('W')` — a correct alternative algorithm that replaces the two-pointer structure.

Fig. 2. A deterministic post-hoc illustration of the measured locality contrast (example `p02597:u054825571:S3`). It is the largest Answer-minus-Progress TED gap among jointly repaired examples whose displayed programs are at most 900 characters; it is excluded from every estimate.

declared frontier; unlike the Seen contrast, this interval also includes zero. The problem-held-out evidence therefore supports portfolio coverage over zero-shot, not relation-specific coverage or a generalized constrained advantage.

Answer to RQ5. The 45 versus 13 discordant pairs establish ZPDPatch's paired advantage over Zero-shot without same-problem training. Because trajectory availability, not randomized difficulty, defines Unseen, its higher absolute RR does not imply that novelty helps repair.

A post-review matching audit makes that caveat testable. Without using repair outcomes, minimum-cost assignment pairs all 54 Unseen problems to distinct Seen problems on current pass rate, statement and code size, AST size, history length, and test count. For current-only A_3, problem-balanced RR is 81.8% on Unseen versus 86.4% on matched Seen: −4.58 points [−14.18, 5.30]. The interval is wide, but the apparent raw reversal disappears; we use only within-split paired contrasts for method claims.

Independent-test confirmation. With all prompt feedback and online selection recomputed from the observed partition, ZPDPatch repairs 192/250 inputs (76.8%) and 189 of those same returned patches also pass every hidden test: joint RR 75.6% [69.9, 80.5] and a 98.4% [95.5, 99.5] hidden-confirmation rate. The selection-matched A_9 control repairs 186 observed and 184 jointly, for 73.6% joint RR [67.8, 78.7] and a 98.9% [96.2, 99.7] confirmation rate. The paired counts are 13 ZPDPatch-only versus eight A_9-only joint repairs (exact McNemar $p = .383$); the equal-problem joint-RR difference is +2.55 points [−0.86, 6.39]. Therefore both methods' observed repairs almost always survive untouched tests, but this confirmation does not distinguish their coverage.

6.6 RQ6: Robustness and External Replication

Conformance, dependence, and seeds. No construction, non-regression, budget, or parse violation occurs. Across three P–S–A runs, Seen/Unseen RR SD is 1.0/1.9; all six fixed-candidate orders preserve RR, while P–S–A minimizes calls and TED. These are conformance checks.

Seen hidden tests and training-target overlap. ZPDPatch/A_9 observed-only RR is 46.3/48.0% and joint hidden RR is 45.3/47.0%; the equal-problem contrast is -0.6 points [-3.23, 2.10]. Exact same-problem training-AC AST matches occur in 0.6/1.3% of generated selections. This rejects widespread exact copying, not memorization.

Scale and independent-domain replication. At 1.5B, $A_3 - A_1 = 13.4$ [11.16, 15.92], while $M_9 - A_9 = -0.8$ [-2.62, 1.07]. The four-exercise Java holdout is positive (3.9 points [2.99, 5.11]), but 151 users cross its boundary; the user-disjoint holdout is null (0.6 points [-1.20, 2.59]). Breadth replicates; relation composition does not.

Problem-cross-fitted primary mechanism estimate. Five fixed folds exclude test-problem identities during selection and give $M_9 - A_9 = 2.2$ points [-0.19, 4.63]; the secondary equal-problem interval includes zero. The six-budget mean remains 1.2 points [0.10, 2.33].

Prompt-distribution control. Removing history improves frozen mixed/Answer RR by 8.8/11.1 Seen points and 7.6/6.0 Unseen points; all CIs exclude zero. Current-only A_3 reaches 72.0/81.2% Seen/Unseen RR. The nominal current-only M_9 selects exactly those three Answer members, so we recommend current-only A_3 for unrestricted deployment.

Verdict-order model sensitivity. Collapsing non-AC verdicts and retraining changes Progress RR by 2.3 points [0.20, 4.48] on Seen and -2.8 [-6.77, 1.19] on Unseen; all Strict intervals include zero. Verdict ordering has no portable benefit.

7 Conclusion

Candidate breadth supplies the largest stable unrestricted gain; current-only Answer-3Seed is therefore the tested default, and mixed-target pools are not justified for unrestricted deployment. Trajectory mining has a narrower role: Progress yields modestly smaller joint repairs, and mixed targets can improve small-budget coverage. The certified controller deploys either choice without observed-test regression or over-budget changes. Whether locality benefits users, or relation effects recur in larger user-disjoint populations, remains unmeasured.

8 Threats to Validity

Construct validity. Passing generated tests is not semantic equivalence, although independent hidden partitions confirm 97.8–98.9% of observed repairs. TED and source retention measure structural change, not readability, hint quality, or learning. We consequently report a Pareto cost axis rather than claiming pedagogical superiority. *Internal validity.* $M_9 - A_9$ changes the complete relation-specific data construction, including target distance and sample count; it does not isolate relation semantics. The original $A_3 - S_3$ contrast also changes decoding, which motivates the added stochastic three-checkpoint cell and temperature sweep. Repository freezing preceded these additions, but none of the analyses was externally preregistered. *Leakage and selection.* Unseen excludes every training problem [Allamanis 2019]; Seen permits problem and user overlap. Hidden-test, cross-fit, exact-AST, token-similarity, and difficulty-matching audits reduce specific alternatives but cannot eliminate algorithmic memorization. We treat the instance-weighted cross-fit null as primary and equal-problem or normalized TED results as sensitivity analyses. *External validity and ethics.* CodeNet users are not a classroom cohort; only CodeWorkout identifies students. Evidence covers one Python model family, one smaller scale, and two Java splits; the positive four-exercise split has user overlap. Public identifiers are anonymized, and no user study validates patch utility.

9 Data Availability

The anonymized replication package supplies splits, checkpoints, JSONL, code, tests, pinned environments, and the `ARTIFACT.md` claim/digest map. Finalization records the exact Git commit, UTC timestamp, file hashes, and row counts in a machine-readable evidence manifest and fails on checkout or result drift.

References

Vincent Aleven and Kenneth R Koedinger. 2000. Limitations of student control: Do students know when they need help?. In *International conference on intelligent tutoring systems*. Springer, 292–303.

Miltiadis Allamanis. 2019. The Adverse Effects of Code Duplication in Machine Learning Models of Code. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. 143–153. doi:10.1145/3359591.3359735

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021). doi:10.48550/arXiv.2107.03374

Dongwook Choi, Jinseok Heo, and Eunseok Lee. 2021. Automated Feedback Generation for Multiple Function Programs. In *2021 28th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 582–583.

Dongwook Choi and Eunseok Lee. 2025. Automated Feedback Generation for Programming Assignments Through Diversification. In *2025 IEEE/ACM 37th International Conference on Software Engineering Education and Training (CSEE&T)*. IEEE, 230–241.

Irene-Angelica Chounta, Patricia Albacete, Pamela Jordan, Sandra Katz, and Bruce M McLaren. 2017. The "Grey Area": A computational approach to model the Zone of Proximal Development. In *European conference on technology enhanced learning*. Springer, 3–16.

Michael Cole, Vera John-Steiner, Sylvia Scribner, and Ellen Souberman. 1978. Mind in society. *Mind in society the development of higher psychological processes. Cambridge, MA: Harvard University Press* (1978).

Albert T Corbett and John R Anderson. 1994. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User modeling and user-adapted interaction* 4, 4 (1994), 253–278.

Zhenlong Dai, Zhuoluo Zhao, Hengning Wang, Xiu Tang, Sai Wu, Chang Yao, Zhipeng Gao, and Jingyuan Chen. 2026. Learner-Tailored Program Repair: A Solution Generator with Iterative Edit-Driven Retrieval Enhancement. *arXiv preprint arXiv:2601.08545* (2026).

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. *Advances in neural information processing systems* 36 (2023), 10088–10115.

Zhangqi Duan, Nigel Fernandez, Alexander Hicks, and Andrew Lan. 2025. Test Case-Informed Knowledge Tracing for Open-ended Coding Tasks. In *Proceedings of the 15th International Learning Analytics and Knowledge Conference*. 235–250. doi:10.1145/3706468.3706500

Bradley Efron. 1979. Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics* 7, 1 (1979), 1–26. doi:10.1214/aos/1176344552

Luca Gazzola, Daniela Micucci, and Leonardo Mariani. 2019. Automatic Software Repair: A Survey. *IEEE Transactions on Software Engineering* 45, 1 (2019), 34–67. doi:10.1109/TSE.2017.2755013

Aritra Ghosh, Neil Heffernan, and Andrew S Lan. 2020. Context-aware attentive knowledge tracing. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 2330–2339.

Elena L. Glassman, Jeremy Scott, Rishabh Singh, Philip J. Guo, and Robert C. Miller. 2015. OverCode: Visualizing Variation in Student Solutions to Programming Problems at Scale. *ACM Transactions on Computer-Human Interaction* 22, 2, Article 7 (2015), 35 pages. doi:10.1145/2699751

Sumit Gulwani, Ivan Radiček, and Florian Zuleger. 2018. Automated clustering and program repair for introductory programming assignments. *ACM SIGPLAN Notices* 53, 4 (2018), 465–480.

Md Mahim Anjum Haque, Wasi Uddin Ahmad, Ismini Lourentzou, and Chris Brown. 2023. Fixeval: Execution-based evaluation of program fixes for programming problems. In *2023 IEEE/ACM International Workshop on Automated Program Repair (APR)*. IEEE, 11–18.

John Hattie and Helen Timperley. 2007. The Power of Feedback. *Review of Educational Research* 77, 1 (2007), 81–112. doi:10.3102/003465430298487

Hasnain Heickal and Andrew Lan. 2024. Generating feedback-ladders for logical errors in programming using large language models. *arXiv preprint arXiv:2405.00302* (2024).

Jinseok Heo, Hohyeon Jeong, Dongwook Choi, and Eunseok Lee. 2023. Referent: Transformer-based feedback generation using assignment information for programming course. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*. IEEE, 101–106.

Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70. doi:10.2307/4615733

Yang Hu, Umair Z Ahmed, Sergey Mechtaev, Ben Leong, and Abhik Roychoudhury. 2020. Re-factoring based program repair applied to programming assignments. In *Proceedings-2019 34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019*. IEEE, 388–398.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2.5-Coder Technical Report. *arXiv preprint arXiv:2409.12186* (2024).

Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of Code Language Models on Automated Program Repair. In *Proceedings of the 45th International Conference on Software Engineering*. 1430–1442. doi:10.1109/ICSE48619.2023.00125

Harshit Joshi, José Cambronero Sanchez, Sumit Gulwani, Vu Le, Gust Verbruggen, and Ivan Radiček. 2023. Repair is nearly generation: Multilingual program repair with llms. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 5131–5140.

Hieke Keuning, Johan Jeuring, and Bastiaan Heeren. 2018. A systematic literature review of automated feedback generation for programming exercises. *ACM Transactions on Computing Education (TOCE)* 19, 1 (2018), 1–43.

Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. 2017. Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles. In *Advances in Neural Information Processing Systems*, Vol. 30.

Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2012. GenProg: A Generic Method for Automatic Software Repair. *IEEE Transactions on Software Engineering* 38, 1 (2012), 54–72. doi:10.1109/TSE.2011.104

Fang Liu, Zhenwei Liu, Qianhui Zhao, Jing Jiang, Li Zhang, Zian Sun, Ge Li, Zhongqi Li, and Yuchi Ma. 2024. Fastfixer: An efficient and effective approach for repairing programming assignments. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 669–680.

Thibaud Lutellier, Hung Viet Pham, Lawrence Pang, Yitong Li, Moshi Wei, and Lin Tan. 2020. CoCoNuT: Combining Context-Aware Neural Translation Models Using Ensemble for Program Repair. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 101–114. doi:10.1145/3395363.3397369

Quinn McNemar. 1947. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika* 12, 2 (1947), 153–157. doi:10.1007/BF02295996

Martin Monperrus. 2018. Automatic Software Repair: A Bibliography. *Comput. Surveys* 51, 1, Article 17 (2018), 24 pages. doi:10.1145/3105906

Tom Murray and Ivon Arroyo. 2002. Toward measuring and maintaining the zone of proximal development in adaptive instructional systems. In *International conference on intelligent tutoring systems*. Springer, 749–758.

Susanne Narciss. 2008. Feedback strategies for interactive learning tasks. In *Handbook of research on educational communications and technology*. Routledge, 125–143.

Pedro Orvalho, Mikoláš Janota, and Vasco M Manquinho. 2025. Counterexample guided program repair using zero-shot learning and maxsat-based fault localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 649–657.

Mateusz Pawlik and Nikolaus Augsten. 2016. Tree edit distance: Robust and memory-efficient. *Information Systems* 56 (2016), 157–173. doi:10.1016/j.is.2015.08.004

Tung Phung, José Cambronero, Sumit Gulwani, Tobias Kohn, Rupak Majumdar, Adish Singla, and Gustavo Soares. 2023. Generating high-precision feedback for programming syntax errors using large language models. *arXiv preprint arXiv:2302.04662* (2023).

Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas J Guibas, and Jascha Sohl-Dickstein. 2015a. Deep knowledge tracing. *Advances in neural information processing systems* 28 (2015).

Chris Piech, Jonathan Huang, Andy Nguyen, Mike Phulsuksombati, Mehran Sahami, and Leonidas Guibas. 2015b. Learning Program Embeddings to Propagate Feedback on Student Code. In *Proceedings of the 32nd International Conference on Machine Learning*. 1093–1102.

Thomas W Price, Yihuan Dong, and Dragan Lipovac. 2017. iSnap: towards intelligent tutoring in novice programming environments. In *Proceedings of the 2017 ACM SIGCSE Technical Symposium on computer science education*. 483–488.

Ruchir Puri, David S Kung, Geert Janssen, Wei Zhang, Giacomo Domeniconi, Vladmir Zolotov, Julian Dolby, Jie Chen, Mihir Choudhury, Lindsey Decker, et al. 2021. Project codenet: A large-scale ai for code dataset for learning a diversity of coding tasks. *arXiv preprint arXiv:2105.12655* 1035 (2021).

Yuhua Qi, Xiaoguang Mao, Yan Lei, Ziying Dai, and Chengsong Wang. 2014. The Strength of Random Search on Automated Program Repair. In *Proceedings of the 36th International Conference on Software Engineering*. 254–265. doi:10.1145/2568225.2568254

Kelly Rivers and Kenneth R. Koedinger. 2017. Data-Driven Hint Generation in Vast Solution Spaces: A Self-Improving Python Programming Tutor. *International Journal of Artificial Intelligence in Education* 27, 1 (2017), 37–64. doi:10.1007/s40593-015-0070-z

Lianne Roest, Hieke Keuning, and Johan Jeuring. 2024. Next-step hint generation for introductory programming using large language models. In *Proceedings of the 26th Australasian Computing Education Conference*. 144–153.

Nadine Schulze Bernd and Irene-Angelica Chounta. 2024. Beyond the grey area: exploring the effectiveness of scaffolding as a learning measure. In *International Conference on Artificial Intelligence in Education*. Springer, 365–378.

Valerie J. Shute. 2008. Focus on Formative Feedback. *Review of Educational Research* 78, 1 (2008), 153–189. doi:10.3102/0034654307313795

Edward K. Smith, Earl T. Barr, Claire Le Goues, and Yuriy Brun. 2015. Is the Cure Worse than the Disease? Overfitting in Automated Program Repair. In *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering*. 532–543.

doi:10.1145/2786805.2786825

Michele Tufano, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. 2019. An Empirical Study on Learning Bug-Fixing Patches in the Wild via Neural Machine Translation. *ACM Transactions on Software Engineering and Methodology* 28, 4, Article 19 (2019), 29 pages. doi:10.1145/3340544

Ke Wang, Rishabh Singh, and Zhendong Su. 2018. Search, align, and repair: data-driven feedback generation for introductory programming exercises. In *Proceedings of the 39th ACM SIGPLAN conference on programming language design and implementation*. 481–495.

Westley Weimer, ThanhVu Nguyen, Claire Le Goues, and Stephanie Forrest. 2009. Automatically Finding Patches Using Genetic Programming. In *Proceedings of the 31st International Conference on Software Engineering*. 364–374. doi:10.1109/ICSE.2009.5070536

Edwin B. Wilson. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *J. Amer. Statist. Assoc.* 22, 158 (1927), 209–212. doi:10.1080/01621459.1927.10502953

Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. 2022. Model Soups: Averaging Weights of Multiple Fine-Tuned Models Improves Accuracy without Increasing Inference Time. In *Proceedings of the 39th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 162)*. 23965–23998.

Chunqiu Steven Xia and Lingming Zhang. 2022. Less Training, More Repairing Please: Revisiting Automated Program Repair via Zero-Shot Learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 959–971. doi:10.1145/3540250.3549101

Ruiwei Xiao, Xinying Hou, and John Stamper. 2024. Exploring how multiple levels of GPT-generated programming hints support or disappoint novices. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. 1–10.

Boyang Yang, Haoye Tian, Weiguo Pian, Haoran Yu, Haitao Wang, Jacques Klein, Tegawendé F Bissyandé, and Shunfu Jin. 2024. Cref: An llm-based conversational software repair framework for programming tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 882–894.

Michihiro Yasunaga and Percy Liang. 2021. Break-it-fix-it: Unsupervised learning for program repair. In *International conference on machine learning*. PMLR, 11941–11952.

He Ye, Matias Martinez, Xiapu Luo, Tao Zhang, and Martin Monperrus. 2022. Selfapr: Self-supervised program repair with test execution diagnostics. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.

Jooyong Yi, Umair Z Ahmed, Amey Karkare, Shin Hwei Tan, and Abhik Roychoudhury. 2017. A feasibility study of using automated program repair for introductory programming assignments. In *Proceedings of the 2017 11th joint meeting on foundations of software engineering*. 740–751.

Jialu Zhang, José Cambronero, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. 2022. Repairing bugs in python assignments using large language models. *arXiv preprint arXiv:2209.14876* (2022).

Jialu Zhang, José Pablo Cambronero, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. 2024. Pydex: Repairing bugs in introductory python assignments using llms. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (2024), 1100–1124.

Kaizhong Zhang and Dennis Shasha. 1989. Simple Fast Algorithms for the Editing Distance Between Trees and Related Problems. *SIAM J. Comput.* 18, 6 (1989), 1245–1262. doi:10.1137/0218082

Qianhui Zhao, Fang Liu, Li Zhang, Yang Liu, Zhen Yan, Zhenghao Chen, Yufei Zhou, Jing Jiang, and Ge Li. 2024. Peer-aided repairer: Empowering large language models to repair advanced student assignments. *arXiv preprint arXiv:2404.01754* (2024). doi:10.48550/arXiv.2404.01754